Bounce Tech Security Review

Auditors

0xjuaan

0xSpearmint

March 28, 2026

Introduction

Obsidian Audits

Obsidian audits is a team of top-ranked security researchers, with a publicly proven track record, specialising in DeFi protocols across EVM chains and Solana.

The team has achieved 10+ top-2 placements in audit competitions, placing 1st in competitions for Wormhole, Pump.fun, Yearn Finance, and many more.

Find out more: obsidianaudits.com

Audit Overview

Bounce Tech is a leveraged token protocol, built on HyperEVM.

Bounce Tech engaged Obsidian Audits to perform a security review of an update to the smart contracts in the `bounce-contracts` repo to support unified account mode for the LeveragedToken smart contract. The review was conducted on March 28, 2026.

Scope

Files in scope

Repo	Files in scope
`bounce-contracts` **Commit hash:** 525def44d0c28a1d51cb5b3221bac6f02cb5d043	PR #192, PR #193, PR #194, PR #195, PR #196

Summary of Findings

Severity Breakdown

A total of **3** issues were identified and categorized based on severity:

- **1 Critical severity**
- **1 Medium severity**
- **1 Informational**

Findings Overview

ID	Title	Severity	Status
C-01	`hyperliquidUsdc` excludes perp account value	Critical	Fixed
M-01	Checkpoint in bridgeToEvm can block mints and bridging	Medium	Fixed
I-01	Missing balance check in bridgeToEvm	Informational	Fixed

Severity Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High -** leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium -** leads to a moderate material loss of assets in the protocol or moderately harms a group of users. Alternatively, breaking a core aspect of the protocol's intended functionality
- **Low -** leads to a minor material loss of assets in the protocol or harms a small group of users.

Likelihood

- **High -** attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium -** the attack/vulnerability requires minimal or no preconditions, but there is limited or no incentive to exploit it in practice
- **Low -** requires highly unlikely precondition states, or requires a significant attacker capital with little or no incentive.

Findings

[C-01] `hyperliquidUsdc` excludes perp account value

Description

In unified account mode, `SpotBalance.total` excludes USDC allocated as margin for perp positions. The perp-side value is tracked separately in `AccountMarginSummary.accountValue`.

The migration removed `perpUsdc()` from `hyperliquidUsdc()`, dropping the entire perp account value from the `totalAssets()` calculation:

```
// Before
function hyperliquidUsdc(address user_) external view override returns
(uint256) {
    return spotUsdc(user_) + perpUsdc(user_);
}

// After
function hyperliquidUsdc(address user_) external view override returns
(uint256) {
    return spotUsdc(user_);
}
```

Since `totalAssets()` drives `exchangeRate()`, which prices every mint and redemption, the result is:

- Exchange rate collapses proportionally to how much capital is deployed in perps
- New minters receive far too many LT tokens, massively diluting existing holders

Example: LT has 10,000 USDC on core, agent opens a perp with 5,000 margin. `totalAssets` reports 5,000 instead of 10,000.

Recommendation

Restore `perpUsdc()` and add it back to `hyperliquidUsdc()`, to ensure that `totalAssets()` returns the correct value.

```
--- a/src/HyperliquidHandler.sol
+++ b/src/HyperliquidHandler.sol
```

```
@@ -17,7 +17,14 @@
     function hyperliquidUsdc(address user_) external view override returns
(uint256) {
-        return spotUsdc(user_);
+        return spotUsdc(user_) + perpUsdc(user_);
     }

     function spotUsdc(address user_) public view override returns
(uint256) {
@@ -24,6 +31,13 @@
         return uint256(spotBalance_.total).scale(_SPOT_DECIMALS,
_USDC_DECIMALS);
     }

+    function perpUsdc(address user_) public view override returns
(uint256) {
+        PrecompileLib.AccountMarginSummary memory accountMarginSummary_ =
+            PrecompileLib.accountMarginSummary(_USDC_PERP_DEX_INDEX,
user_);
+        if (accountMarginSummary_.accountValue < 0) return 0;
+        return uint256(int256(accountMarginSummary_.accountValue));
+    }
+
     function marginUsedUsdc(address user_) public view override returns
(uint256) {
```

```
--- a/src/interfaces/IHyperliquidHandler.sol
+++ b/src/interfaces/IHyperliquidHandler.sol
@@ -6,6 +6,8 @@
     function spotUsdc(address user_) external view returns (uint256);

+    function perpUsdc(address user_) external view returns (uint256);
+
     function marginUsedUsdc(address user_) external view returns
(uint256);
```

Remediation: Fixed in commit d157da3

[M-01] Checkpoint in bridgeToEvm can block mints and bridging

Description

When all USDC is on Core, `_checkpoint` computes a non-zero streaming fee and attempts to transfer it from the contract's EVM-side balance, which is zero. This causes the call to revert.

`bridgeToEvm` calls `_checkpoint`, so it reverts whenever the contract holds no USDC on the EVM side. The mint function (other pathway for increasing the EVM balance) also calls `_checkpoint` and reverts for the same reason. Both paths to increasing EVM-side USDC are blocked, forcing a direct token donation to the contract to avoid the revert.

Recommendation

Consider removing the `_checkpoint` call from `bridgeToEvm` in `LeveragedToken`

```
  function bridgeToEvm(uint256 amount_) external override onlyAgents {
      if (amount_ == 0) revert InvalidAmount();
-     _checkpoint();
      CoreWriterLib.bridgeToEvm(address(_baseAsset()), amount_);
      emit BridgeToEvm(msg.sender, amount_);
  }
```

Remediation: Fixed in commit 4df881d

[I-01] Missing balance check in bridgeToEvm

Description

`bridgeToCore` validates that the requested amount does not exceed the EVM-side USDC balance before executing the bridge.

However `bridgeToEvm` performs no equivalent check against the Core-side balance.

As a result, calls to `bridgeToEvm()` may succeed on the EVM while silently failing due to insufficient balance on HyperCore.

Recommendation

Consider adding a balance check in `bridgeToEvm` before calling `CoreWriterLib.bridgeToEvm`, matching the pattern used by `bridgeToCore` for its EVM-side check.

```
  function bridgeToEvm(uint256 amount_) external override onlyAgents {
      if (amount_ == 0) revert InvalidAmount();
      _checkpoint();
+     LeveragedTokenStorage storage $ = _getLeveragedTokenStorage();
+     if (amount_ > $.globalStorage.hyperliquidHandler().spotUsdc(address(this))) revert InsufficientBalance();
      CoreWriterLib.bridgeToEvm(address(_baseAsset()), amount_);
      emit BridgeToEvm(msg.sender, amount_);
  }
```

Remediation: Fixed in commit 6f320a9